OPENCODE · DEVELOPER GUIDE

OPENCODE
완벽 가이드

설치부터 엔터프라이즈 운영까지

터미널에서 만나는 오픈소스 AI 코딩 에이전트

참조 · opencode.ai/docs/ko

지원 모델 · Claude, GPT, Gemini, Copilot 및 75개 이상의 LLM 공급자

OPENCODE · DEVELOPER GUIDE

O P E N C O D E

완벽 가이드

설치부터 엔터프라이즈 운영까지

목 차

제1부 · 시작하기

01 OpenCode 소개 7

02 사전 준비와 설치 10

03 Quickstart — 첫 세션 만들기 15

04 TUI 명령어 전체 정리 22

제2부 · 공급자와 모델

01 공급자 소개 — 75개 이상 LLM 지원 27

02 OpenCode Zen / Go — 검증된 모델 구독 31

03 주요 공급자 연결 가이드 33

04 로컬 모델 설정 (Ollama, LM Studio) 38

제3부 · 사용법

01 질문하기 — @ 파일 참조와 ! 셸 명령 43

02 Plan / Build Mode — Tab 키 전환 46

03 변경 되돌리기 — /undo, /redo 49

04 세션 관리와 공유 51

제4부 · 확장과 자동화

01 MCP 서버 설정 54

02 Custom Commands — 슬래시 명령 64

03 Agents — 전문 에이전트 구성 67

04 Tools / Skills 71

제5부 · 환경 설정

01 Config — 전역 / 프로젝트 설정 77

02 AGENTS.md — 프로젝트 컨텍스트 82

03 TUI 설정 — 테마 · 키바인드 · 알림 85

04 권한과 보안 88

제6부 · 운영과 활용

01 GitHub / GitLab 연동 91

02 Enterprise 구성 93

03 문제 해결 (Troubleshooting) 96

04 마무리 — 강점과 활용 추천 100

PART 1 · CHAPTER 1

01

OpenCode 소개

터미널에서 만나는
오픈소스 AI 코딩 에이전트

OpenCode란?

OpenCode는 Anomaly가 공개한 오픈소스 AI 코딩 에이전트입니다. 터미널 기반 인터페이스(TUI), 데스크톱 앱, IDE 확장 프로그램으로 사용할 수 있습니다.

- **오픈소스** · GitHub `anomalyco/opencode` — 160,000개 이상의 스타
- **무료 모델 포함** · 별도 API 구독 없이 시작 가능
- **다중 공급자** · Claude, GPT, Gemini, Copilot 등 75개 이상의 LLM
- **멀티 플랫폼** · macOS, Windows (WSL 권장), Linux
- **월간 7.5M 사용자** · 900명 기여자, 13,000개 이상의 커밋

핵심 기능

기능	설명
LSP 지원	LLM에 적합한 LSP 자동 로드
멀티 세션	동일 프로젝트에서 여러 에이전트 병렬 실행
링크 공유	참조·디버깅용 세션 링크 공유
GitHub Copilot	GitHub 로그인으로 Copilot 계정 사용
ChatGPT Plus/Pro	OpenAI 로그인으로 ChatGPT 계정 사용
모든 모델	Models.dev로 로컬 포함 75개+ 공급자 지원
모든 에디터	TUI · 데스크톱 앱 · IDE 확장

인터페이스 종류

인터페이스	설명	적합한 상황
TUI	터미널 사용자 인터페이스	SSH 환경, 가장 일반적
CLI	명령줄 일회 실행	스크립트, 자동화
Web	브라우저 기반 UI	원격 협업, 데모
IDE	IDE 확장 프로그램	에디터와 함께 사용
데스크톱	macOS/Windows/Linux 앱	독립 실행 환경

프라이버시 우선 설계

OpenCode는 코드나 컨텍스트 데이터를 외부 서버에 저장하지 않습니다. 입력한 자격증명은 로컬 파일 `~/.local/share/opencode/auth.json`에만 저장됩니다.

> OpenCode는 매달 **7.5M명** 이상의 개발자가 사용하며, **160,000개** 이상의 GitHub 스타와 **900명**의 기여자로 빠르게 성장하는 오픈소스 프로젝트입니다.

OpenCode 사용 흐름

1. **설치** · 설치 스크립트, npm, Homebrew 등으로 빠르게 설치
2. **공급자 연결** · `/connect` 명령으로 LLM 공급자 연결
3. **모델 선택** · `/models`로 사용할 모델 선택
4. **프로젝트 초기화** · `/init`으로 AGENTS.md 생성
5. **대화 시작** · 자연어로 질문하거나 `@`로 파일 참조

PART 1 · CHAPTER 2

02

사전 준비와 설치

시스템 요구사항부터
OS별 설치 방법까지

사전 준비

터미널에서 OpenCode를 사용하려면 다음이 필요합니다.

최신 터미널 에뮬레이터

- **WezTerm** · 크로스 플랫폼
- **Alacritty** · 크로스 플랫폼, 고성능
- **Ghostty** · Linux 및 macOS
- **Kitty** · Linux 및 macOS

LLM 공급자의 API 키

OpenCode Zen, Anthropic, OpenAI, Google 등 사용할 공급자의 자격증명이 필요합니다. 처음이라면 OpenCode Zen이 가장 간단합니다.

설치 — 가장 빠른 방법

설치 스크립트를 사용하면 OS와 아키텍처를 자동 감지하여 설치합니다.

```
curl -fsSL https://opencode.ai/install | bash
```

Node.js 사용

```
# npm
npm install -g opencode-ai

# Bun
bun install -g opencode-ai

# pnpm
pnpm install -g opencode-ai

# Yarn
yarn global add opencode-ai
```

macOS / Linux — Homebrew

```
brew install anomalyco/tap/opencode
```

권장 tap

최신 릴리스를 사용하려면 OpenCode tap 사용을 권장합니다. 공식 `brew install opencode` formula는 Homebrew 팀이 관리하며 업데이트 주기가 더 깁니다.

Arch Linux

```
# Stable (공식 저장소)
sudo pacman -S opencode

# Latest (AUR)
paru -S opencode-bin
```

Windows — WSL 권장

Windows에서는 **Windows Subsystem for Linux (WSL)** 사용을 강력히 권장합니다. 성능이 더 좋고 OpenCode 기능과의 완전한 호환성을 제공합니다.

```
# Chocolatey
choco install opencode

# Scoop
scoop install opencode

# npm
npm install -g opencode-ai

# Mise
mise use -g github:anomalyco/opencode

# Docker
docker run -it --rm ghcr.io/anomalyco/opencode
```

환경 차이

Windows 네이티브 환경에서는 일부 TUI 명령어가 표시되지 않거나 동작이 다를 수 있습니다. 가능하면 WSL 사용을 권장합니다.

바이너리 직접 설치

패키지 매니저가 어려운 환경에서는 GitHub Releases에서 바이너리를 직접 받아 PATH에 두는 방법도 있습니다.

```
https://github.com/anomalyco/opencode/releases
```

설치 확인

```
opencode --version
```

인증 — 공급자 연결

설치 후 `opencode`를 실행하면 TUI가 열립니다. TUI에서 `/connect` 명령으로 공급자를 연결합니다.

```
/connect
```

처음이라면 **OpenCode Zen**을 추천합니다. OpenCode 팀이 테스트하고 검증한 모델 목록을 제공합니다.

1. `/connect` 실행 → OpenCode Zen 선택
2. `opencode.ai/auth`로 이동 → 로그인
3. 결제 정보 입력 후 API 키 복사
4. API 키를 터미널에 붙여넣기

> **자격증명 저장 위치**
>
> 입력한 자격증명은 `~/.local/share/opencode/auth.json`에 저장됩니다.
> `opencode auth list`로 연결된 공급자를 확인할 수 있습니다.

PART 1 · CHAPTER 3

03

Quickstart

첫 세션 만들기

STEP 1 — 프로젝트 폴더로 이동하고 실행

OpenCode는 현재 작업 디렉토리를 기준으로 동작합니다. 작업할 프로젝트로 이동한 뒤 `opencode`를 실행합니다.

```
cd /path/to/project
opencode
```

또는 디렉토리 경로를 인자로 지정할 수도 있습니다.

```
opencode /path/to/project
```

STEP 2 — /connect로 공급자 연결

TUI에서 `/connect`를 실행하고 공급자를 선택합니다. 처음이라면 **OpenCode Zen**을 권장합니다.

```
/connect
```

STEP 3 — /models로 모델 선택

```
/models
```

STEP 4 — /init으로 프로젝트 초기화

`/init`을 실행하면 OpenCode가 프로젝트를 분석하고 프로젝트 루트에 `AGENTS.md` 파일을 생성합니다.

```
/init
```

권장

생성된 `AGENTS.md` 파일은 Git에 커밋하는 것을 권장합니다.

STEP 5 — 질문하기와 @ 파일 참조

프롬프트에 바로 질문을 입력합니다. `@` 키로 파일을 퍼지(fuzzy) 검색하여 컨텍스트로 추가할 수 있습니다.

```
Give me a quick summary of the codebase.

How is authentication handled in @packages/functions/src/api/
index.ts?
```

STEP 6 — 기능 추가는 Plan Mode부터

새 기능을 추가할 때는 먼저 **Tab** 키를 눌러 Plan Mode로 전환하세요. Plan Mode에서는 파일을 수정하지 않고 구현 방법만 제안합니다.

```
<TAB>  ← Plan Mode

When a user deletes a note, we'd like to flag it as
deleted in the database. Then create a screen that shows
all the recently deleted notes.
```

계획이 마음에 들면 **Tab** 키를 다시 눌러 Build Mode로 전환하고 변경을 적용해 달라고 요청합니다.

```
<TAB>  ← Build Mode

Sounds good! Go ahead and make the changes.
```

STEP 7 — 바로 변경하기

비교적 간단한 변경은 계획 없이 바로 구현하도록 요청할 수 있습니다.

```
We need to add authentication to the /settings route.
Take a look at how this is handled in
```

```
@packages/functions/src/notes.ts and implement the same
logic in @packages/functions/src/settings.ts
```

STEP 8 — 변경 되돌리기

OpenCode가 적용한 변경이 마음에 들지 않으면 `/undo` 로 되돌립니다.

```
/undo
```

다시 적용하려면 `/redo` 를 실행합니다.

```
/redo
```

> **Git 저장소 필요**
>
> `/undo` 와 `/redo` 는 내부적으로 Git을 사용합니다. 프로젝트가 Git 저장소여야 하며, Windows 네이티브 환경에서는 이 명령이 표시되지 않을 수 있습니다.

STEP 9 — 세션 공유

`/share` 를 실행하면 현재 대화 링크가 생성되어 클립보드에 복사됩니다.

```
/share
```

Quickstart 요약

STEP	명령	목적
1	`opencode`	TUI 실행
2	`/connect`	공급자 연결
3	`/models`	모델 선택
4	`/init`	AGENTS.md 생성
5	`@파일명`	파일 참조
6	**Tab**	Plan/Build 전환
7	바로 요청	간단한 변경
8	`/undo`	되돌리기
9	`/share`	공유

PART 1 · CHAPTER 4

04

TUI 명령어 전체 정리

공식 문서 기준
슬래시 명령어 Cheatsheet

공식 슬래시 명령어 목록

OpenCode TUI에서 `/` 뒤에 명령 이름을 입력합니다. 대부분의 명령에는 기본 리더 키 `ctrl+x` 키바인드도 있습니다.

> **환경 차이**
>
> 아래 명령어들은 공식 문서의 표준 목록입니다. OpenCode 버전이나 환경(특히 Windows 네이티브)에 따라 일부 명령이 표시되지 않을 수 있으니, 현재 환경의 명령은 `/help` 로 확인하세요.

명령	설명	키바인드
`/connect`	공급자 추가 및 API 키 설정	—
`/models`	사용 가능한 모델 목록	`ctrl+x m`
`/init`	AGENTS.md 생성/업데이트	`ctrl+x i`
`/new`	새 세션 (별칭 /clear)	`ctrl+x n`
`/sessions`	세션 목록·전환	`ctrl+x l`
`/compact`	세션 압축 (별칭 /summarize)	`ctrl+x c`
`/undo`	마지막 메시지 실행 취소	`ctrl+x u`
`/redo`	실행 취소 다시 실행	`ctrl+x r`
`/share`	현재 세션 공유	`ctrl+x s`
`/unshare`	세션 공유 취소	—
`/editor`	외부 편집기로 작성	`ctrl+x e`
`/export`	Markdown으로 내보내기	`ctrl+x x`
`/themes`	테마 목록	`ctrl+x t`
`/thinking`	추론 블록 가시성 토글	—
`/details`	도구 실행 세부 정보 토글	`ctrl+x d`
`/help`	도움말 표시	`ctrl+x h`
`/exit`	종료 (별칭 /quit, /q)	`ctrl+x q`

특수 입력 문법

문법	기능	예시
`@파일명`	퍼지 파일 검색	`@index.ts`
`!명령`	셸 명령 실행	`!ls -la`
`<TAB>`	Plan ↔ Build 전환	우하단 모드 표시 확인

그룹별 정리

세션 시작과 종료

- `/new` · 새 세션
- `/sessions` · 세션 목록
- `/exit` · 종료

공급자와 모델

- `/connect` · 새 공급자 추가
- `/models` · 모델 변경

변경 관리

- `/undo` · 변경 되돌리기 (Git 필요)
- `/redo` · 다시 적용
- `/compact` · 컨텍스트 절약

공유와 내보내기

- `/share` · 링크 공유
- `/unshare` · 공유 취소
- `/export` · Markdown 저장

설정과 도움말

- `/themes` · 테마 변경
- `/editor` · 외부 편집기
- `/help` · 현재 환경 명령 확인

PART 2 · CHAPTER 1

01

공급자 소개

75개 이상의 LLM 공급자 지원

공급자 개요

OpenCode는 **AI SDK**와 **Models.dev**를 사용하여 75개 이상의 LLM 공급자를 지원합니다. Claude, GPT, Gemini, Copilot 등 주요 AI 모델을 하나의 인터페이스에서 사용할 수 있습니다.

공급자 연결 기본 절차

1. TUI에서 `/connect` 명령 실행
2. 목록에서 원하는 공급자 선택
3. API 키 또는 OAuth 인증 완료
4. `/models`로 사용할 모델 선택

주요 공급자 목록

공급자	특징	인증 방식
Anthropic	Claude Pro/Max 또는 API	OAuth / API 키
OpenAI	ChatGPT Plus/Pro 또는 API	OAuth / API 키
GitHub Copilot	Copilot 구독 활용	GitHub OAuth
Google Vertex AI	GCP 프로젝트 필요	gcloud / 서비스 계정
Amazon Bedrock	AWS 자격증명 필요	AWS 인증 체인
DeepSeek	저렴한 API 비용	API 키
OpenRouter	통합 게이트웨이	API 키
Ollama	로컬 모델 실행	불필요
Groq	초고속 추론	API 키
GitLab Duo	GitLab 구독 활용	OAuth / PAT

자격증명은 `~/.local/share/opencode/auth.json` 에 저장됩니다. `opencode auth list` 로 연결된 공급자를 확인하세요.

공급자 설정 (opencode.json)

```
{
  "$schema": "https://opencode.ai/config.json",
  "model": "anthropic/claude-sonnet-4-5",
  "provider": {
    "anthropic": {
      "options": {
        "timeout": 600000,
        "apiKey": "{env:ANTHROPIC_API_KEY}"
      }
    }
  }
}
```

공급자 허용 / 차단

```
{
  // 특정 공급자만 허용
  "enabled_providers": ["anthropic", "openai"],

  // 또는 특정 공급자 차단
  "disabled_providers": ["openai", "gemini"]
}
```

우선순위

`disabled_providers` 가 `enabled_providers` 보다 우선합니다.

small_model 설정

제목 생성 같은 가벼운 작업에는 별도의 작은 모델을 사용합니다.

```
{
  "model": "anthropic/claude-sonnet-4-5",
  "small_model": "anthropic/claude-haiku-4-5"
}
```

PART 2 · CHAPTER 2

02

OpenCode Zen / Go

검증된 모델 구독 서비스

OpenCode Zen이란?

OpenCode Zen은 OpenCode 팀이 직접 테스트하고 벤치마킹한 엄선된 AI 모델 세트에 액세스를 제공하는 구독 서비스입니다.

Zen 연결 방법

1. `/connect` → OpenCode Zen 선택
2. `opencode.ai/auth`로 이동, 로그인 및 결제
3. API 키 복사 후 TUI에 붙여넣기
4. `/models`로 추천 모델 확인

```
/connect

/models
```

OpenCode Go란?

OpenCode Go는 인기 있는 오픈 코딩 모델에 안정적으로 접근할 수 있는 **저렴한 구독 요금제**입니다.

연결 절차는 Zen과 동일하며, `/connect`에서 **OpenCode Go**를 선택합니다.

> **처음이라면 추천**
>
> LLM 공급자를 처음 사용한다면 OpenCode Zen으로 시작하는 것을 권장합니다.

Zen / Go 비교

구분	Zen	Go
대상	모든 사용자	비용 절감 중시
제공 모델	프런티어 모델	오픈 코딩 모델
요금제	사용량 기반	저가 구독

PART 2 · CHAPTER 3

03

주요 공급자 연결

Anthropic, OpenAI, Copilot,
Vertex AI, Bedrock 설정

Anthropic (Claude)

1. `/connect` 실행 → Anthropic 선택
2. **Claude Pro/Max** 구독이 있으면 OAuth 인증
3. 또는 API Key 직접 입력

```
# 환경 변수로 설정
export ANTHROPIC_API_KEY="sk-ant-..."

# opencode.json에서 참조
{
  "provider": {
    "anthropic": {
      "options": { "apiKey": "{env:ANTHROPIC_API_KEY}" }
    }
  }
}
```

OpenAI (ChatGPT Plus/Pro)

1. `/connect` → OpenAI 선택
2. **ChatGPT Plus/Pro**로 OAuth 진행
3. 또는 API 키 직접 입력

GitHub Copilot

1. `/connect` → GitHub Copilot 검색
2. `github.com/login/device` → 코드 입력
3. `/models` 로 모델 선택

> **참고**
>
> 일부 모델(GPT-4o, Claude 등)은 GitHub Copilot Pro+ 구독이 필요할 수 있습니다.

Google Vertex AI

```
# 1. gcloud CLI 인증
gcloud auth application-default login

# 2. 환경 변수 설정
export GOOGLE_CLOUD_PROJECT="your-project-id"
export VERTEX_LOCATION="global"

# 3. 실행
opencode

# 4. 모델 선택
/models
```

Amazon Bedrock

```
# 방법 1: 환경 변수
AWS_ACCESS_KEY_ID=XXX \
AWS_SECRET_ACCESS_KEY=YYY \
opencode

# 방법 2: Named Profile
AWS_PROFILE=my-profile opencode

# 방법 3: Bearer Token
AWS_BEARER_TOKEN_BEDROCK=XXX opencode
```

```
{
  "provider": {
    "amazon-bedrock": {
      "options": {
        "region": "us-east-1",
        "profile": "my-aws-profile"
      }
    }
  }
}
```

모델 액세스

Amazon Bedrock 콘솔의 Model catalog에서 원하는 모델에 대한 액세스를 먼저 요청해야 합니다.

OpenRouter

OpenRouter는 하나의 API 키로 여러 공급자의 모델을 사용할 수 있는 통합 게이트웨이입니다.

1. OpenRouter 대시보드에서 API 키 생성
2. `/connect` → OpenRouter 검색
3. API 키 입력
4. `/models` 로 모델 선택

DeepSeek

1. DeepSeek 콘솔에서 API Key 생성
2. `/connect` → DeepSeek 선택
3. API 키 입력 후 `/models`

Groq

Groq는 자체 LPU로 초고속 추론을 제공합니다.

1. Groq 콘솔에서 API Key 생성
2. `/connect` → Groq 검색 → 키 입력
3. `/models` 에서 모델 선택

GitLab Duo

1. `/connect` → GitLab 선택
2. OAuth(권장) 또는 PAT 인증
3. `/models` 로 Duo 모델 선택

PART 2 · CHAPTER 4

04

로컬 모델 설정

Ollama, LM Studio,
llama.cpp 연결

Ollama

Ollama를 사용하면 로컬에서 오픈소스 LLM을 실행하고 OpenCode에 연결할 수 있습니다.

```
{
  "$schema": "https://opencode.ai/config.json",
  "provider": {
    "ollama": {
      "npm": "@ai-sdk/openai-compatible",
      "name": "Ollama (local)",
      "options": {
        "baseURL": "http://localhost:11434/v1"
      },
      "models": {
        "llama3.1": { "name": "Llama 3.1" },
        "deepseek-coder": { "name": "DeepSeek Coder" }
      }
    }
  }
}
```

도구 호출 문제

도구 호출이 작동하지 않으면 Ollama의 `num_ctx` 를 16k~32k로 늘려보세요. Qwen-Coder나 DeepSeek-Coder 모델을 권장합니다.

LM Studio

```
{
  "provider": {
    "lmstudio": {
      "npm": "@ai-sdk/openai-compatible",
      "name": "LM Studio (local)",
      "options": {
        "baseURL": "http://127.0.0.1:1234/v1"
      },
      "models": {
        "google/gemma-3n-e4b": {
          "name": "Gemma 3n (local)"
```

```
          }
        }
      }
    }
  }
```

llama.cpp

```
{
  "provider": {
    "llama.cpp": {
      "npm": "@ai-sdk/openai-compatible",
      "name": "llama-server (local)",
      "options": {
        "baseURL": "http://127.0.0.1:8080/v1"
      },
      "models": {
        "qwen3-coder:a3b": {
          "name": "Qwen3-Coder a3b-30b (local)",
          "limit": {
            "context": 128000,
            "output": 65536
          }
        }
      }
    }
  }
}
```

사용자 정의 공급자

OpenAI 호환 API를 제공하는 모든 공급자를 추가할 수 있습니다.

```
{
  "provider": {
    "myprovider": {
      "npm": "@ai-sdk/openai-compatible",
      "name": "My AI Provider",
      "options": {
        "baseURL": "https://api.myprovider.com/v1",
        "apiKey": "{env:MY_API_KEY}"
      },
      "models": {
        "my-model": {
          "name": "My Model",
          "limit": { "context": 200000, "output": 65536 }
        }
      }
```

```
      }
    }
  }
```

PART 3 · CHAPTER 1

01

질문하기

@ 파일 참조와
! 셸 명령

코드베이스 이해 요청

OpenCode에 코드베이스 설명을 요청할 수 있습니다. 처음 보는 코드나 직접 작업하지 않은 영역을 이해할 때 특히 유용합니다.

```
Give me a quick summary of the codebase.

What does the authentication flow look like?

Explain how the payment system is structured.
```

@ 키로 파일 참조

메시지 입력 중 `@`를 입력하면 현재 작업 디렉토리에서 파일을 퍼지(fuzzy) 검색합니다.

```
How is auth handled in @packages/functions/src/api/index.ts?

Compare the implementation of @src/notes.ts
and @src/settings.ts and tell me the differences.
```

활용 팁

`@`는 정확한 경로를 외울 필요 없이 파일명 일부만 입력해도 자동 완성됩니다. 여러 파일을 한 번에 참조하면 OpenCode가 코드 간 관계까지 파악합니다.

! 셸 명령 실행

메시지를 `!`로 시작하면 셸 명령을 실행하고 결과를 대화에 추가합니다.

```
!ls -la
!git log --oneline -10
!cat package.json
```

이미지 첨부

이미지를 터미널에 드래그 앤 드롭하여 프롬프트에 추가할 수 있습니다.

```
We'd like to design this new screen using a design I've used
before.

[Image #1] Take a look at this image and use it as a reference.
```

좋은 프롬프트 작성법

좋지 않은 예

```
Fix the bug.
```

좋은 예

```
The login form in @src/components/Login.tsx shows
"undefined" instead of the actual error message when
the user enters wrong credentials.

The error handling should match what we do in
@src/components/Signup.tsx — show a user-friendly
message and clear the password field.
```

코드베이스 탐색 패턴

- **전체 구조** · "Give me a quick summary of the codebase"
- **특정 기능** · "How is X implemented?"
- **의존 관계** · "What does @file.ts depend on?"
- **패턴 파악** · "What patterns does this project use for X?"
- **변경 영향** · "If I change @file.ts, what will break?"

PART 3 · CHAPTER 2

02

Plan / Build Mode

Tab 키로 계획 모드와
구현 모드 전환

두 가지 모드

OpenCode TUI는 **Plan Mode**와 **Build Mode** 두 가지 모드로 동작합니다. **Tab** 키로 전환하며 화면 오른쪽 하단에서 현재 모드를 확인합니다.

모드	동작	용도
Plan Mode	파일 수정 비활성화	계획 검토, 코드베이스 이해
Build Mode	파일 읽기/쓰기 활성화	실제 코드 수정 및 구현

STEP 1 — Plan Mode로 계획 만들기

새 기능 추가는 먼저 계획을 만드는 것을 권장합니다. **Tab** 키를 눌러 Plan Mode로 전환한 뒤 원하는 작업을 자세히 설명합니다.

```
<TAB>  ← Plan Mode

When a user deletes a note, we'd like to flag it as
deleted in the database. Then create a screen that shows
all the recently deleted notes. From this screen, the user
can undelete a note or permanently delete it.
```

컨텍스트가 핵심

OpenCode가 원하는 작업을 이해할 수 있도록 충분한 세부 정보를 제공해야 합니다. 팀의 주니어 개발자에게 말하듯이 설명하면 도움이 됩니다.

STEP 2 — 계획 다듬기

계획이 나오면 피드백을 주거나 이미지를 첨부하여 디자인 참고 자료를 추가할 수 있습니다.

```
We'd like to design this new screen using a design
I've used before.

[Image #1] Take a look at this image and use it as
a reference.
```

STEP 3 — Build Mode로 전환하여 구현

```
<TAB>  ← Build Mode로 전환

Sounds good! Go ahead and make the changes.
```

모드 전환 흐름

```
[Build Mode]  ← 시작
       ↓ Tab
[Plan Mode]   ← 계획·설계·탐색
       ↓ Tab
[Build Mode]  ← 실제 구현
```

모드 확인

모드를 잘못 인지하고 변경을 요청하면 Plan Mode에서 파일이 수정되지 않거나, 의도치 않게 Build Mode에서 파일이 변경될 수 있습니다. 항상 화면 우하단 모드 표시를 확인하세요.

PART 3 · CHAPTER 3

03

변경 되돌리기

/undo와 /
redo로
안전한 AI 코딩

/undo 명령

OpenCode에 변경을 요청했다고 해보겠습니다.

```
Can you refactor the function in @packages/functions/src/api/
index.ts?
```

원하는 결과가 아니었다면 `/undo` 명령으로 변경을 되돌릴 수 있습니다.

```
/undo
```

OpenCode가 방금 적용한 변경을 되돌리고 원래 메시지를 다시 보여줍니다. 가장 최근의 사용자 메시지, 모든 후속 응답, 모든 파일 변경 사항이 함께 제거됩니다.

연속 실행

`/undo` 는 여러 번 연속으로 실행할 수 있습니다.

/redo 명령

```
/redo
```

Git 저장소 필요

`/undo` 와 `/redo` 는 내부적으로 Git을 사용하여 파일 변경 사항을 관리합니다. 프로젝트가 **Git 저장소**여야 하며, Windows 네이티브 환경에서는 이 명령이 표시되지 않거나 동작이 다를 수 있습니다.

키바인드

- `/undo` · `ctrl+x u`
- `/redo` · `ctrl+x r`

PART 3 · CHAPTER 4

04

세션 관리와 공유

/new · /sessions · /compact
/share · /unshare

새 세션 — /new

대화가 길어져 컨텍스트가 복잡해지거나 완전히 다른 주제로 작업을 시작할 때는 `/new` 로 새 세션을 시작합니다.

```
/new   # 별칭: /clear
```

세션 목록과 전환 — /sessions

```
/sessions   # 별칭: /resume, /continue
```

세션 압축 — /compact

세션이 길어져 컨텍스트가 가득 차가면 `/compact` 로 압축할 수 있습니다.

```
/compact   # 별칭: /summarize
```

세션 공유 — /share

```
/share
```

공유를 취소하려면 `/unshare` 를 사용합니다.

```
/unshare
```

> **기본값**
> 대화는 기본적으로 공유되지 않습니다. `opencode.json` 에서 `"share": "auto"` 또는 `"disabled"` 로 변경할 수 있습니다.

대화 내보내기 — /export

현재 대화를 Markdown 파일로 내보내고 기본 편집기에서 열 수 있습니다.

```
/export
```

```
# EDITOR 환경변수 예시
export EDITOR=nano
export EDITOR=vim
export EDITOR="code --wait"
```

외부 편집기로 작성 — /editor

```
/editor
```

도구 실행 세부 정보 — /details

```
/details
```

추론 블록 표시 — /thinking

```
/thinking
```

> **추론 기능 자체 토글**
>
> `/thinking` 은 표시 여부만 제어합니다. 실제 추론 기능을 토글하려면 `ctrl+t` 로 모델 변형을 순환하세요.

PART 4 · CHAPTER 1

01

MCP 서버 설정

로컬 및 원격 MCP로
OpenCode 기능 확장

MCP란?

Model Context Protocol (MCP)는 AI 모델이 외부 도구·데이터 소스와 안전하게 통신할 수 있게 하는 개방형 프로토콜입니다. OpenCode는 MCP 서버를 통해 브라우저 자동화, 데이터베이스 조작, 이슈 트래커 연동 등 외부 시스템에 접근할 수 있습니다.

MCP 서버를 추가하면 그 서버가 제공하는 도구들이 OpenCode의 내장 도구와 함께 LLM에 노출되어, 별도 코드 없이도 자연어로 외부 시스템을 다룰 수 있습니다.

설정 파일 위치

MCP 서버는 `opencode.json` 파일의 `mcp` 섹션에 정의합니다. 어느 위치에 두느냐에 따라 적용 범위가 달라집니다.

위치	적용 범위	용도
`~/.config/opencode/opencode.json`	전역 (모든 프로젝트)	개인 공통 도구
프로젝트 루트의 `opencode.json`	해당 프로젝트만	프로젝트별 도구, 팀 공유
`.well-known/opencode`	조직 전체 (Remote)	회사 기본값

> **팀 공유**
> 프로젝트 루트의 `opencode.json` 을 Git에 커밋하면 팀 전체가 동일한 MCP 도구 환경을 자동으로 사용할 수 있습니다.

MCP 서버 유형

유형	설명	예시
`local`	로컬 프로세스 실행 (stdio)	Playwright, Filesystem
`remote`	원격 HTTP/SSE 엔드포인트	Supabase, Sentry

로컬 MCP 서버

로컬 MCP 서버는 OpenCode가 자식 프로세스로 실행하는 MCP 구현체입니다. `type`을 `"local"`로 설정하고, `command`에 실행 명령을 배열로 지정합니다.

Playwright 예시 — 브라우저 자동화

```
{
  "$schema": "https://opencode.ai/config.json",
  "mcp": {
    "playwright": {
      "type": "local",
      "command": ["npx", "-y", "@playwright/mcp@latest"],
      "enabled": true
    }
  }
}
```

이 설정으로 OpenCode는 브라우저를 직접 조작할 수 있게 됩니다. 페이지 열기, 클릭, 폼 입력, 스크린샷 캡처, 콘솔 로그 읽기 등을 자연어로 요청할 수 있습니다.

필드 설명

- `type` · `"local"` 고정
- `command` · 실행 명령과 인자 배열 (첫 요소는 실행 파일)
- `enabled` · 서버 활성화 여부 (기본값 true)
- `environment` · 자식 프로세스에 전달할 환경변수 (선택)

환경변수 전달 예시

```
{
  "mcp": {
    "filesystem": {
      "type": "local",
      "command": ["npx", "-y",
                  "@modelcontextprotocol/server-filesystem",
                  "/path/to/allowed/dir"],
      "enabled": true,
```

```
      "environment": {
        "DEBUG": "mcp:*",
        "API_KEY": "{env:MY_API_KEY}"
      }
    }
  }
}
```

변수 치환

`environment` 의 값에 `{env:VAR}` 또는 `{file:~/path}` 형식을 쓰면 시스템 환경변수나 파일 내용을 참조할 수 있습니다.

원격 MCP 서버

원격 MCP 서버는 HTTP/SSE 엔드포인트를 통해 통신합니다. `type`을 `"remote"`로 설정하고 `url`을 지정합니다.

Supabase 예시 — 데이터베이스 연동

```
{
  "$schema": "https://opencode.ai/config.json",
  "mcp": {
    "supabase": {
      "type": "remote",
      "url": "https://mcp.supabase.com/mcp",
      "enabled": true
    }
  }
}
```

이 설정으로 OpenCode는 Supabase 프로젝트의 데이터베이스 스키마 조회, 쿼리 실행, 마이그레이션 관리 등을 수행할 수 있습니다.

OAuth 인증이 필요한 경우

```
{
  "mcp": {
    "sentry": {
      "type": "remote",
      "url": "https://mcp.sentry.dev/mcp",
      "enabled": true,
      "oauth": {}
    }
  }
}
```

`oauth: {}`를 추가하면 OpenCode가 OAuth 흐름을 자동 처리합니다. 첫 사용 시 브라우저가 열리고 인증을 완료하면 토큰이 안전하게 저장됩니다.

헤더 추가 (커스텀 인증)

```
{
  "mcp": {
    "my-api": {
      "type": "remote",
      "url": "https://api.example.com/mcp",
      "enabled": true,
      "headers": {
        "Authorization": "Bearer {env:MY_API_TOKEN}",
        "X-Custom-Header": "value"
      }
    }
  }
}
```

보안 주의

API 토큰을 `opencode.json` 에 직접 적지 마세요. `{env:VAR}` 또는 `{file:~/path}` 로 환경변수나 별도 파일에서 읽어오는 것을 권장합니다.

MCP 서버 인증 관리

OAuth가 필요한 MCP 서버는 CLI 명령으로 인증 상태를 관리할 수 있습니다.

```
# 특정 서버 인증
opencode mcp auth supabase

# 모든 MCP 서버와 인증 상태 확인
opencode mcp list

# 저장된 자격증명 제거 (재인증 필요시)
opencode mcp logout supabase

# 연결 디버깅 — 통신 로그 확인
opencode mcp debug supabase
```

인증 토큰은 `~/.local/share/opencode/mcp-auth.json`에 안전하게 저장됩니다.

MCP 도구 활성화 / 비활성화

MCP 서버가 제공하는 도구 정의는 컨텍스트에 추가됩니다. 도구가 많은 서버를 여러 개 활성화하면 컨텍스트가 빠르게 소진되므로, `tools` 섹션에서 Glob 패턴으로 필요한 도구만 선택적으로 활성화하는 것이 좋습니다.

전역 비활성화 후 특정 에이전트에서만 활성화

```
{
  "tools": {
    "playwright*": false
  },
  "agent": {
    "qa-tester": {
      "tools": {
        "playwright*": true
      }
    }
  }
}
```

특정 도구만 비활성화

```
{
  "tools": {
    "supabase_delete_*": false,
    "supabase_execute_sql": false
  }
}
```

도구 이름 확인

`opencode mcp list` 로 각 서버가 제공하는 도구 이름을 확인할 수 있습니다. 일반적으로 `{서버이름}_{도구이름}` 형식입니다.

여러 MCP 서버 조합 예시

실무에서는 여러 MCP 서버를 함께 사용하는 경우가 많습니다. 다음은 풀스택 웹 개발 환경의 예시입니다.

```
{
  "$schema": "https://opencode.ai/config.json",
  "mcp": {
    "playwright": {
      "type": "local",
      "command": ["npx", "-y", "@playwright/mcp@latest"],
      "enabled": true
    },
    "supabase": {
      "type": "remote",
      "url": "https://mcp.supabase.com/mcp",
      "enabled": true
    },
    "context7": {
      "type": "remote",
      "url": "https://mcp.context7.com/mcp",
      "enabled": true
    },
    "sentry": {
      "type": "remote",
      "url": "https://mcp.sentry.dev/mcp",
      "enabled": true,
      "oauth": {}
    }
  }
}
```

유용한 MCP 서버

서버	유형	용도
Playwright	local	브라우저 자동화, E2E 테스트
Supabase	remote	데이터베이스, 인증, 스토리지
Context7	remote	라이브러리 문서 실시간 검색
Sentry	remote	오류 추적, 이슈 관리
Grep by Vercel	remote	GitHub 코드 스니펫 검색
GitHub	remote	이슈, PR, 리포지토리 관리
Filesystem	local	지정 디렉토리 파일 접근
Notion	remote	문서, 위키 통합

권한 설정 확인

MCP 서버 중에는 파일을 직접 쓰거나 외부 시스템을 변경하는 도구가 있습니다. `permission` 설정으로 실행 전 승인을 받도록 하는 것이 안전합니다 (제5부 04 권한과 보안 참조).

PART 4 · CHAPTER 2

02

Custom Commands

반복 작업을 위한
사용자 정의 슬래시 명령

Custom Command 설정

반복적인 작업을 위한 커스텀 슬래시 명령을 만들 수 있습니다.

```
{
  "command": {
    "test": {
      "template": "Run the full test suite with coverage report and
show any failures.\nFocus on failing tests and suggest fixes.",
      "description": "Run tests with coverage",
      "agent": "build",
      "model": "anthropic/claude-haiku-4-5"
    },
    "component": {
      "template": "Create a new React component named $ARGUMENTS
with TypeScript.\nInclude proper typing and basic structure.",
      "description": "Create a new component"
    }
  }
}
```

사용법

```
/test               # 전체 테스트 실행
/component Button   # Button 컴포넌트 생성
```

$ARGUMENTS

`$ARGUMENTS` 를 template에 포함하면 명령 실행 시 추가 인자를 전달할 수 있습니다.

Markdown 파일로 정의

```
# .opencode/commands/review.md
---
description: Code review checklist
agent: build
---

Review the changes in $ARGUMENTS for:
```

```
- Security issues
- Performance concerns
- Code style consistency
- Test coverage gaps
```

PART 4 · CHAPTER 3

03

Agents

전문 에이전트 구성으로
작업별 최적화

에이전트 구성

특정 작업에 최적화된 전문 에이전트를 구성할 수 있습니다.

```
{
  "agent": {
    "code-reviewer": {
      "description": "Reviews code for best practices",
      "model": "anthropic/claude-sonnet-4-5",
      "prompt": "You are a code reviewer. Focus on security,
performance, and maintainability.",
      "tools": {
        "write": false,
        "edit": false
      }
    }
  }
}
```

내장 에이전트

에이전트	역할
`build`	기본 구현 에이전트 (파일 수정 가능)
`plan`	계획 전용 에이전트 (읽기 전용)

기본 에이전트 변경

```
{
  "default_agent": "plan"
}
```

Markdown으로 에이전트 정의

`~/.config/opencode/agents/` 또는 `.opencode/agents/`의 Markdown 파일로도 에이전트를 정의할 수 있습니다. 파일은 **프론트매터(YAML)**로 메타데이터를 지정하고, 본문에 시스템 프롬프트를 작성합니다.

예시: 백엔드 개발자 에이전트

```
# .opencode/agents/backend-developer.md
---
name: backend-developer
mode: subagent
description: 확장 가능한 API 개발과 마이크로서비스 아키텍처를
  전문으로 하는 시니어 백엔드 엔지니어. 성능, 보안,
  유지보수성에 중점을 둔 견고한 서버 측 솔루션을 구축합니다.
---

당신은 서버 측 애플리케이션을 전문으로 하는 시니어
백엔드 개발자이며, Node.js 18+, Python 3.11+, Go 1.21+에
깊은 전문성을 보유하고 있습니다. 당신의 주요 임무는
확장 가능하고 안전하며 성능이 뛰어난 백엔드 시스템을
구축하는 것입니다.
```

프론트매터 주요 필드

필드	설명
`name`	에이전트 식별자 (파일명과 일치 권장)
`mode`	`primary` (기본) 또는 `subagent` (다른 에이전트가 호출)
`description`	에이전트 역할 요약 — 자동 호출 판단에 사용됨
`model`	사용할 모델 (선택)
`tools`	허용/차단할 도구 목록 (선택)

description의 중요성

`description` 은 다른 에이전트가 이 에이전트를 자동으로 호출할지 판단하는 기준이 됩니다. 역할과 전문 영역을 구체적으로 작성하세요.

PART 4 · CHAPTER 4

04

Tools / Skills

도구 권한 관리와
에이전트 스킬 활용

내장 도구 (Built-in Tools)

OpenCode는 여러 내장 도구를 LLM에 제공합니다. `tools` 섹션에서 활성화/비활성화 할 수 있습니다.

```
{
  "$schema": "https://opencode.ai/config.json",
  "tools": {
    "write": false,
    "bash": false
  }
}
```

도구	역할
`read`	파일 읽기
`write`	새 파일 작성
`edit`	기존 파일 수정
`bash`	셸 명령 실행
`grep`	코드 검색
`glob`	파일 패턴 매칭

에이전트별 도구 설정

에이전트별로 다른 도구 세트를 적용하려면 `agent` 섹션의 각 에이전트 안에 `tools` 를 정의합니다. 예를 들어 리뷰 전용 에이전트에는 `write: false, edit: false` 로 읽기 전용 환경을 만들 수 있습니다.

에이전트 스킬 (Skills)

스킬(Skill)은 특정 작업에 필요한 노하우와 절차를 모아둔 폴더 단위 패키지입니다. `SKILL.md` 파일에 언제 사용할지·언제 사용하지 않을지·어떤 절차를 따를지 명시하면, OpenCode가 작업의 성격에 맞춰 자동으로 해당 스킬을 활용합니다.

스킬 폴더 구조

스킬은 `.opencode/skills/` (프로젝트) 또는 `~/.config/opencode/skills/` (전역) 디렉토리 아래 각각의 폴더로 정의합니다.

```
.opencode/skills/
├── backend-architect/
│   └── SKILL.md
├── react-component/
│   └── SKILL.md
└── api-endpoint/
    └── SKILL.md
```

각 폴더의 `SKILL.md` 는 프론트매터(YAML)로 메타데이터를 지정하고, 본문에 시스템 프롬프트와 작업 절차를 작성합니다.

스킬 작성 원칙

- **역할 명확화** · 어떤 전문 영역의 작업을 다루는지 한 줄로
- **사용 조건 명시** · "이때 사용", "이때 사용 안 함"을 구체적으로
- **절차 단계화** · 1, 2, 3 형태의 명확한 순서
- **맥락 정보** · 도메인 용어, 비기능 요건, 제약사항 포함

예시: backend-architect 스킬

다음은 백엔드 아키텍처 설계를 다루는 스킬의 전체 예시입니다. "Use / Do not use" 섹션을 명시했기 때문에 OpenCode가 현재 작업의 성격을 판단해 적절히 호출합니다 — 작은 버그 수정에는 사용되지 않고, 새 서비스 설계 요청에는 자동으로 사용됩니다.

```
---
name: backend-architect
description: Expert backend architect specializing
  in scalable API design, microservices architecture,
  and distributed systems.
---
You are a backend system architect specializing in
scalable, resilient, and maintainable backend systems
and APIs.

## Use this skill when

- Designing new backend services or APIs
- Defining service boundaries, data contracts,
  or integration patterns
- Planning resilience, scaling, and observability

## Do not use this skill when

- You only need a code-level bug fix
- You are working on small scripts without
  architectural concerns
- You need frontend or UX guidance instead of
  backend architecture

## Instructions

1. Capture domain context, use cases, and
   non-functional requirements.
2. Define service boundaries and API contracts.
3. Choose architecture patterns and integration
   mechanisms.
4. Identify risks, observability needs, and
   rollout plan.
```

SKILL.md 구성 요소

프론트매터 (YAML)

필드	설명
`name`	스킬 식별자 (폴더명과 일치 권장)
`description`	스킬의 전문 영역 한 줄 요약 — 자동 선택의 핵심 기준

본문 섹션 구조

본문은 자유 형식이지만, 효과적인 스킬은 다음 구조를 따르는 경우가 많습니다.

1. **시스템 프롬프트 선언** · "You are a..." 로 역할 정의
2. **Use this skill when** · 사용해야 할 상황 목록
3. **Do not use this skill when** · 사용하지 말아야 할 상황 목록
4. **Instructions** · 작업 절차를 번호 매긴 단계로

스킬 활용 예시 시나리오

요청 유형	적합한 스킬
"새 결제 서비스 설계하자"	`backend-architect`
"이 React 컴포넌트 만들어줘"	`react-component`
"REST API 엔드포인트 추가"	`api-endpoint`
"여기 오타 하나 고쳐줘"	(스킬 호출 안 됨 — Do not use 조건)

> **description의 중요성**
>
> OpenCode는 스킬의 `description`을 보고 자동 선택 여부를 판단합니다. 모호하게 적으면 적절한 시점에 호출되지 않을 수 있으니, 전문 영역과 강점을 구체적으로 작성하세요.

팀 공유

`.opencode/skills/` 를 Git에 커밋하면 팀 전체가 동일한 작업 방법론을 공유할 수 있습니다. 사내 코딩 규칙, 아키텍처 결정 절차, 리뷰 체크리스트 등을 스킬로 정리하면 효과적입니다.

PART 5 · CHAPTER 1

01

Config 설정

전역·프로젝트 설정 파일
상세 가이드

설정 파일 형식

OpenCode는 **JSON**과 **JSONC**(주석 포함 JSON) 형식을 모두 지원합니다. 설정은 교체가 아니라 **병합**됩니다.

```
{
  "$schema": "https://opencode.ai/config.json",
  "model": "anthropic/claude-sonnet-4-5",
  "autoupdate": true,
  "server": {
    "port": 4096
  }
}
```

설정 파일 위치와 우선순위

1. **Remote** (`.well-known/opencode`) — 조직 기본값
2. **Global** (`~/.config/opencode/opencode.json`)
3. **Custom** (`OPENCODE_CONFIG` env)
4. **Project** (프로젝트 루트의 `opencode.json`)
5. **Inline** (`OPENCODE_CONFIG_CONTENT` env)

주요 Config 옵션

옵션	설명
`model`	기본 모델
`small_model`	경량 작업용 모델
`autoupdate`	자동 업데이트
`share`	세션 공유 기본값
`default_agent`	기본 에이전트
`instructions`	지침 파일 경로
`permission`	도구 권한 설정

변수 치환

```
{
  "model": "{env:OPENCODE_MODEL}",
  "provider": {
    "anthropic": {
      "options": {
        "apiKey": "{env:ANTHROPIC_API_KEY}"
      }
    },
    "openai": {
      "options": {
        "apiKey": "{file:~/.secrets/openai-key}"
      }
    }
  }
}
```

Compaction 설정

```
{
  "compaction": {
    "auto": true,
    "prune": true,
    "reserved": 10000
  }
}
```

Watcher 설정

```
{
  "watcher": {
    "ignore": ["node_modules/**", "dist/**", ".git/**"]
  }
}
```

Sharing 설정

```
{
  "share": "manual"
  // "manual"   - 수동 공유 (기본값)
  // "auto"     - 자동 공유
  // "disabled" - 공유 비활성화
}
```

PART 5 · CHAPTER 2

02

AGENTS.md

프로젝트 컨텍스트와
AI 지침 파일

AGENTS.md란?

`AGENTS.md`는 OpenCode에게 프로젝트 구조, 코딩 패턴, 규칙을 알려주는 파일입니다. `/init` 명령으로 자동 생성되며 Git에 커밋하는 것을 권장합니다.

```
/init
```

AGENTS.md 예시

```
# Project Guidelines

## Tech Stack
- Frontend: React + TypeScript
- Backend: Node.js + Express
- Database: PostgreSQL with Prisma ORM

## Coding Standards
- Use TypeScript strict mode
- Follow ESLint config in .eslintrc
- Write tests for all new functions

## MCP Tools
When you need to search docs, use `context7` tools.
If unsure how to do something, use `gh_grep` to search
GitHub examples.
```

추가 지침 파일

```
{
  "instructions": [
    "CONTRIBUTING.md",
    "docs/guidelines.md",
    ".cursor/rules/*.md"
  ]
}
```

팀 컨텍스트 공유

AGENTS.md를 Git에 커밋하면 팀원 모두가 동일한 AI 컨텍스트로 작업할 수 있습니다.

PART 5 · CHAPTER 3

03

TUI 설정

테마 · 키바인드 · 알림

tui.json 파일

TUI 관련 설정은 `~/.config/opencode/tui.json` 에서 관리합니다.

```
{
  "$schema": "https://opencode.ai/tui.json",
  "theme": "opencode",
  "leader_timeout": 2000,
  "keybinds": {
    "leader": "ctrl+x",
    "command_list": "ctrl+p"
  },
  "scroll_speed": 3,
  "scroll_acceleration": { "enabled": false },
  "diff_style": "auto",
  "mouse": true
}
```

주요 옵션

옵션	설명
`theme`	UI 테마
`leader_timeout`	leader 키 후 대기 시간(ms)
`scroll_speed`	스크롤 속도
`diff_style`	diff 렌더링 (auto, stacked)
`mouse`	마우스 캡처

테마 변경 — /themes

```
/themes
```

Attention (알림)

OpenCode가 답변을 기다리거나 권한 승인이 필요할 때 알림을 받을 수 있습니다.

```
{
  "attention": {
    "enabled": true,
    "notifications": true,
    "sound": true,
    "volume": 0.4,
    "sound_pack": "opencode.default",
    "sounds": {
      "error": "./sounds/error.mp3"
    }
  }
}
```

- `enabled` · 알림 전체 활성화 (기본값 false)
- `notifications` · 데스크톱 알림
- `sound` · 알림음
- `volume` · 0~1

키바인드 커스터마이징

```
{
  "$schema": "https://opencode.ai/tui.json",
  "keybinds": {
    "leader": "ctrl+x",
    "command_list": "ctrl+p"
  }
}
```

키바인드는 내장 기본값과 병합되므로 변경하려는 단축키만 설정하면 됩니다.

PART 5 · CHAPTER 4

04

권한과 보안

도구 사용 승인 방식

기본 동작

OpenCode는 기본적으로 명시적 승인 없이 모든 작업을 허용합니다. `permission` 옵션으로 도구별 승인 동작을 설정할 수 있습니다.

```
{
  "$schema": "https://opencode.ai/config.json",
  "permission": {
    "edit": "ask",
    "bash": "ask"
  }
}
```

권한 모드

모드	동작
`allow`	자동 허용 (기본값)
`ask`	실행 전 사용자 승인 요청
`deny`	실행 차단

권장 설정 패턴

안전 우선 (초보자)

```
{
  "permission": {
    "edit": "ask",
    "write": "ask",
    "bash": "ask"
  }
}
```

CI/CD 자동화

```
{
  "permission": {
    "edit": "allow",
    "bash": "deny"
  }
}
```

셸 실행 주의

신뢰할 수 없는 코드베이스에서는 반드시 `"bash": "ask"` 로 설정하세요.

PART 6 · CHAPTER 1

01

GitHub / GitLab 연동

버전 관리 플랫폼과
OpenCode 통합

GitHub Copilot 연결

```
/connect
# → GitHub Copilot 선택
# → github.com/login/device 코드 입력
```

GitLab Duo 연동

```
/connect
# → GitLab 선택
# → OAuth (권장) 또는 Personal Access Token

/models
# - duo-chat-haiku-4-5  (빠른 작업)
# - duo-chat-sonnet-4-5 (균형 - 권장)
# - duo-chat-opus-4-5   (복잡한 분석)
```

셀프 호스팅 GitLab

```
export GITLAB_INSTANCE_URL=https://gitlab.company.com
export GITLAB_TOKEN=glpat-...

# 사용자 정의 AI Gateway
export GITLAB_AI_GATEWAY_URL=https://ai-gateway.company.com

# 규정 준수 설정
{
  "small_model": "gitlab/duo-chat-haiku-4-5",
  "share": "disabled"
}
```

관리자 활성화

GitLab 관리자가 Duo Agent Platform과 관련 기능 플래그를 활성화해야 합니다.

PART 6 · CHAPTER 2

02

Enterprise 구성

조직 배포와
중앙 집중식 설정

Remote Config — 조직 기본값

조직은 `.well-known/opencode` 엔드포인트로 기본 설정을 제공할 수 있습니다.

```
# 조직 Remote config
# https://company.com/.well-known/opencode
{
  "mcp": {
    "jira": {
      "type": "remote",
      "url": "https://jira.example.com/mcp",
      "enabled": false
    }
  }
}

# 사용자 로컬 설정으로 활성화
{
  "mcp": {
    "jira": {
      "enabled": true
    }
  }
}
```

공급자 제한

```
{
  "enabled_providers": ["anthropic", "openai"],
  "disabled_providers": ["openai", "gemini"]
}
```

프라이버시 설정

```
{
  "share": "disabled",
```

```
  "autoupdate": false
}
```

규정 준수

민감한 환경에서는 세션 공유와 자동 업데이트를 비활성화하세요.

PART 6 · CHAPTER 3

03

문제 해결

Troubleshooting

공급자 연결 문제

```
# 현재 연결된 공급자 확인
opencode auth list

# 사용자 정의 공급자 확인 항목
# 1. /connect 공급자 ID와 opencode.json ID 일치
# 2. 올바른 npm 패키지 사용
# 3. options.baseURL 값 확인
```

자주 발생하는 문제

문제	해결 방법
TUI 화면 깨짐	최신 터미널 사용
한글 입력 깨짐	UTF-8 로케일 확인
로컬 모델 도구 호출 실패	num_ctx를 16k+ 로 설정
Azure OpenAI 오류	콘텐츠 필터를 Default로
컨텍스트 초과	`/compact` 또는 `/new`
공급자 인증 실패	`opencode auth list`
Windows 일부 명령 안 보임	WSL 사용 권장, `/help` 확인

Windows 환경 차이

Windows 네이티브 환경에서는 일부 슬래시 명령어(`/undo` , `/redo` 등)가 표시되지 않거나 동작이 다를 수 있습니다. WSL 사용을 권장합니다.

Windows (WSL) 설정

```
# 1. WSL 활성화 (관리자 PowerShell)
wsl --install

# 2. WSL 내에서 OpenCode 설치
curl -fsSL https://opencode.ai/install | bash

# 3. WSL 내에서 실행
cd /mnt/c/Users/you/projects/myproject
opencode
```

로그와 디버깅

```
# MCP 서버 디버깅
opencode mcp debug

# MCP 인증 상태 확인
opencode mcp auth list

# 자격증명 확인
opencode auth list
```

업데이트와 재설치

```
# npm
npm update -g opencode-ai

# Homebrew
brew upgrade opencode

# 설치 스크립트로 재설치
curl -fsSL https://opencode.ai/install | bash
```

도움 받을 곳

- **공식 문서** · `opencode.ai/docs/ko`

- **Discord** · `opencode.ai/discord`
- **GitHub Issues** · `github.com/anomalyco/opencode/issues`

PART 6 · CHAPTER 4

04

마무리

OpenCode의 강점과
활용 추천

OpenCode의 핵심 강점

- **오픈소스** · 완전 오픈소스, 커뮤니티 기반 발전
- **공급자 자유** · 75개 이상 LLM 공급자
- **기존 구독 활용** · Claude Pro, ChatGPT Plus, GitHub Copilot
- **프라이버시** · 코드를 서버에 저장하지 않음
- **확장성** · MCP, 플러그인, 커스텀 에이전트
- **모든 환경 지원** · TUI / CLI / IDE / 데스크톱 / Web

활용 추천 시나리오

상황	추천 설정
개인 프로젝트	OpenCode Zen + 기본 설정
팀 협업	AGENTS.md 커밋 + `/share`
대규모 코드베이스	Plan Mode 우선
프라이버시 중요	로컬 모델(Ollama)
기업 환경	Remote Config + 공급자 제한
Windows 사용자	WSL 환경 사용 권장

OpenCode는 매달 **7.5M명** 이상의 개발자가 사용하는 빠르게 성장하는 오픈소스 AI 코딩 에이전트입니다. 지금 시작해보세요.

다음 단계

- **공식 문서** · `opencode.ai/docs/ko`
- **SDK** · `opencode.ai/docs/sdk`
- **Custom Commands** · `opencode.ai/docs/commands`
- **변경 내역** · `opencode.ai/ko/changelog`